SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NAME: DENISHA SHREE R

REG NO: 192524016

COURSE CODE: DSA0101

COURSE NAME: C++ PROGRAMMING

DESIGN,IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

Task 1 — Object-Oriented Analysis (4 Marks)

1. Objects and their data and behaviour

Object	Data	Behaviour
Passenger	Name, age, passenger type	Selects a concession
SmartCard	Card ID, balance	Recharge and deduct fare
Journey	Source, destination, time	Calculates journey fare
Zone	Zone name and base fare	Identifies the fare zone
FareTable	Zone fares and surcharge	Provides the correct fare

2. Procedural vs Object-Oriented Programming

Procedural programming uses separate variables and functions. Object-oriented programming groups related data and functions into classes. OOP makes the metro system easier to maintain and extend.

3. OOP principles

- Encapsulation: Protect the smart-card balance using private data.
- Abstraction: Provide a simple function such as payFare().

- Inheritance: Create Student and SeniorCitizen passenger types from a Passenger class.
- Polymorphism: Implement different concession calculations for different passenger types.

4. Rules

- The balance must never become negative.
- At least ₹50 must remain after a journey.
- Peak surcharge applies from 8–10 AM and 5–8 PM, excluding the end times.
- Invalid zones and invalid inputs must be rejected.

Task 2 — Tokens, Keywords, Identifiers and Constants

C++ has five main token types.

Token	Example
Keywords	int, class, if, return
Identifiers	balance, fare, SmartCard
Constants	5000, 20, 'A'
Operators	+, -, *, ==
Punctuators	;, {}, (), ,

Constants

```
#include <iostream>

using namespace std;

constexpr long long INNER_FARE = 1000;

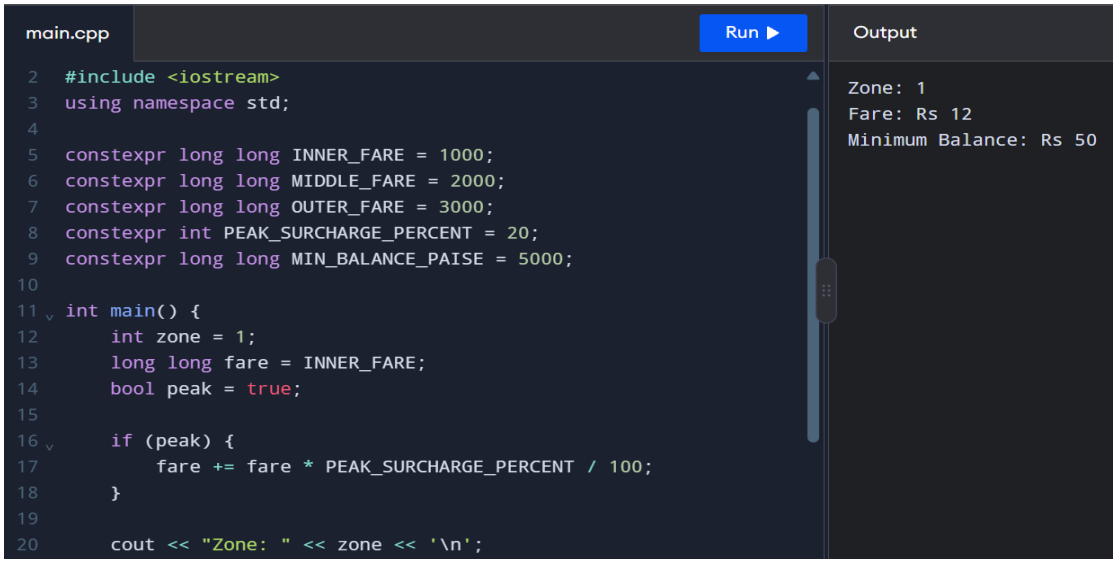
constexpr long long MIDDLE_FARE = 2000;

constexpr long long OUTER_FARE = 3000;

constexpr int PEAK_SURCHARGE_PERCENT = 20;

constexpr long long MIN_BALANCE_PAISE = 5000;
```

```
int main() {
    int zone = 1;
    long long fare = INNER_FARE;
    bool peak = true;
    if (peak) {
        fare += fare * PEAK_SURCHARGE_PERCENT / 100;
    }
    cout << "Zone: " << zone << '\n';
    cout << "Fare: Rs " << fare / 100.0 << '\n';
    cout << "Minimum Balance: Rs "
         << MIN_BALANCE_PAISE / 100.0 << '\n';
    return 0;
}
```



The fares are stored in paise. For example, 1000 paise equals ₹10. #define is a preprocessor macro without normal type checking or scope. const creates a read-only object, while constexpr permits compile-time evaluation when its initializer is a constant expression. enum class is safer than an ordinary enum because its values are scoped and do not implicitly convert to integers.

Complexity: Constant lookup is \(O(1)\). Changing a constant requires updating one definition rather than several fare calculations.

Task 3 — Data Types and Compatibility

Data	Suitable type	Reason
Fare	long long	Stores paise exactly
Balance	long long	Prevents ordinary integer overflow for large balances
Peak status	bool	Stores true or false
Zone	enum class	Represents valid zone names
Hour	int	Stores the hour of a day
Passenger count	long long	Supports large counts
Character	char	Stores one character
Percentage calculation	long long	Supports integer arithmetic

The exact sizes depend on the compiler and system. They can be checked using sizeof().

Why use integer paise instead of double rupees?

Floating-point values can introduce rounding errors. Integer paise represents amounts exactly.

For example:

- ₹10 = 1000 paise
- ₹20 = 2000 paise
- ₹30 = 3000 paise

Type compatibility

- int combined with double generally produces a double.

- Converting a number to bool gives false for zero and true for nonzero values.
- Integer division discards the fractional part.
- Comparing signed and unsigned integers can produce unexpected results.

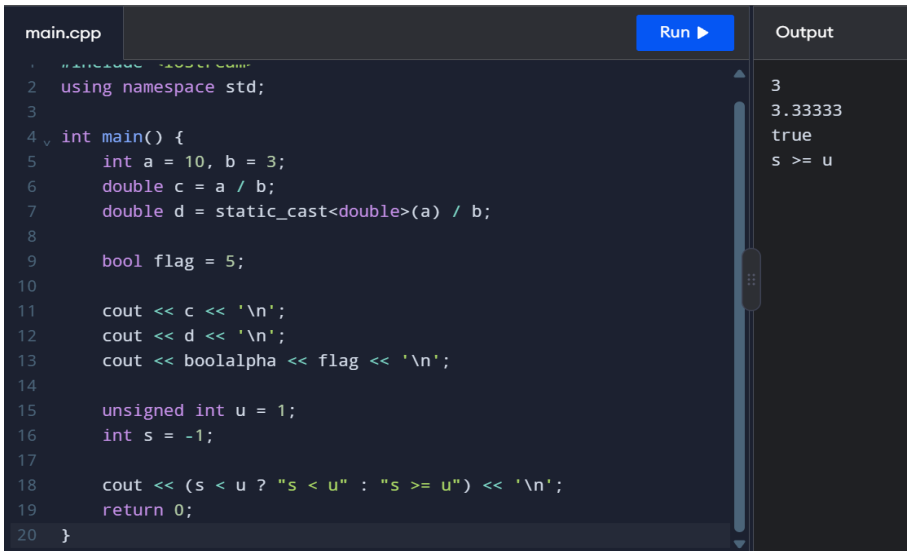
<climits> provides limits such as INT_MAX and INT_MIN. If a network collects more than INT_MAX, a larger type such as long long is needed.

Complexity: Adding one fare takes \(O(1)\) time with either integer paise or double rupees. Both commonly occupy 8 bytes for the selected types, but integer paise avoids floating-point rounding in monetary calculations.

Task 4 — Control Implicit Conversions

Example program:

#include <iostream>using namespace std;
int main() {
int a = 10, b = 3;
double c = a / b;
double d = static_cast<double>(a) / b;
bool flag = 5;
cout << c << '\n';
cout << d << '\n';
cout << boolalpha << flag << '\n';
unsigned int u = 1;
int s = -1;
cout << (s < u ? "s < u" : "s >= u") << '\n';
return 0;
}



Explanation

- a / b performs integer division, producing 3. Assigning it to double does not restore the fraction.
- static_cast<double>(a) / b converts a before division, producing approximately 3.33333.
- bool flag = 5 converts a nonzero integer to true.
- In the signed/unsigned comparison, s is converted to an unsigned value, so the comparison gives false on a typical system with 32-bit unsigned int.

Explicit conversions make the program clearer. They generally do not add significant runtime or memory overhead when the compiler can optimize them.

Task 5 — Operators and Precedence

Operator precedence determines which operation is evaluated first. Parentheses can change the order.

Example:

```
#include <iostream>
using namespace std;
int main() {
    int x = 5, y = 7;
    cout << x + y * 2 << ' '
         << (x + y) * 2 << '\n';
    int z = x++ + +4 + y;
    cout << z << ' ' << x << '\n';
    cout << (x > y ? x : y) << ' '
         << x % 4 * 2 << '\n';
    cout << 10 / 3 << ' '
         << 10 % 3 << ' '
```

```
        << -10 / 3 << ' '
        << -10 % 3 << '\n';
    int k = 0;
    for (int i = 0; i < 10; i += 3)
        k += i;
    cout << k << '\n';
    return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 7;

    cout << x + y * 2 << ' '
         << (x + y) * 2 << '\n';

    int z = x++ + +4 + y;
    cout << z << ' ' << x << '\n';

    cout << (x > y ? x : y) << ' '
         << x % 4 * 2 << '\n';

    cout << 10 / 3 << ' '
         << 10 % 3 << ' '
         << -10 / 3 << ' '
         << -10 % 3 << '\n';
```

```
19 24
16 6
7 4
3 1 -3 -1
18
```

Explanation

- Multiplication occurs before addition: \(5+7\times2=19\).
- Parentheses change the order: \((5+7)\times2=24\).
- `x++` uses the original value before incrementing `x`.
- The conditional operator selects the larger value.
- `%` calculates the remainder.
- Integer division truncates toward zero.
- The loop adds `0 + 3 + 6 + 9 = 18`.

Complexity: Each arithmetic operation is \(O(1)\). The loop runs four times, so it is \(O(1)\) for this fixed limit.

Task 6 — Control Structures and Fare Calculation

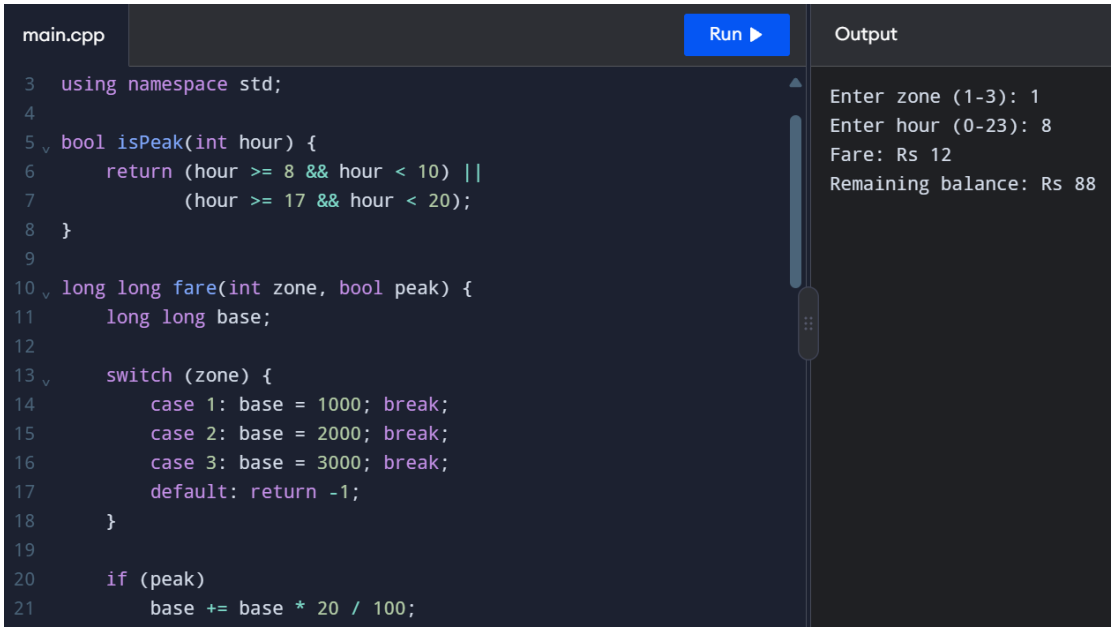
The program uses switch to select a zone fare, if to check peak hours, and if-else to determine whether travel is allowed.

Peak hours are 8:00–10:00 AM and 5:00–8:00 PM. The start is included and the end is excluded.

Example:

```
#include <iostream>
using namespace std;
bool isPeak(int hour) {
   return (hour >= 8 && hour < 10) ||
         (hour >= 17 && hour < 20);}
long long fare(int zone, bool peak) {
   long long base;
   switch (zone) {
      case 1: base = 1000; break;
      case 2: base = 2000; break;
      case 3: base = 3000; break;
      default: return -1;
   }
   if (peak)
      base += base * 20 / 100;
   return base;}
bool canTravel(long long balance, long long f) {
   return f >= 0 && balance - f >= 5000;}
int main() {
   int zone, hour;
   long long balance = 10000;
   cout << "Enter zone (1-3): ";
   cin >> zone;
   cout << "Enter hour (0-23): ";
   cin >> hour;
   if (zone < 1 || zone > 3 || hour < 0 || hour > 23) {
      cout << "Invalid input\n";
      return 1;
   }
   long long f = fare(zone, isPeak(hour));
   if (canTravel(balance, f)) {
      balance -= f;
      cout << "Fare: Rs " << f / 100.0 << '\n';
```

```
    cout << "Remaining balance: Rs "
         << balance / 100.0 << '\n';
} else {
    cout << "Travel denied: Insufficient balance\n";
}
return 0;}
```



Alternative: An array indexed by zone can replace switch. Both provide \(O(1)\) lookup. An array is convenient when adding more zones, while a switch explicitly handles each case.

Task 7 — Procedural vs Object-Oriented Programming

1.Procedural approach:

This approach uses separate variables and functions. It is simple for small programs, but the balance and fare rules are not grouped into one object.

Example:

```
#include <iostream>using namespace std;
long long balance = 10000;
long long calculateFare(int zone, bool peak) {
    long long f = zone * 1000LL;
    if (peak)
        f += f * 20 / 100;
```

```
    return f;}
int main() {
    long long f = calculateFare(1, true);
    if (balance - f >= 5000) {
        balance -= f;
        cout << "Fare: Rs " << f / 100.0 << '\n';
        cout << "Balance: Rs " << balance / 100.0 << '\n';
    } else {
        cout << "Insufficient balance\n";
    }
    return 0;}
```



2. Object-Oriented Programming

The class protects the balance and allows fare payments through a controlled method.

Example:

```
#include <iostream>

using namespace std;

class SmartCard {

private:

    long long balance;

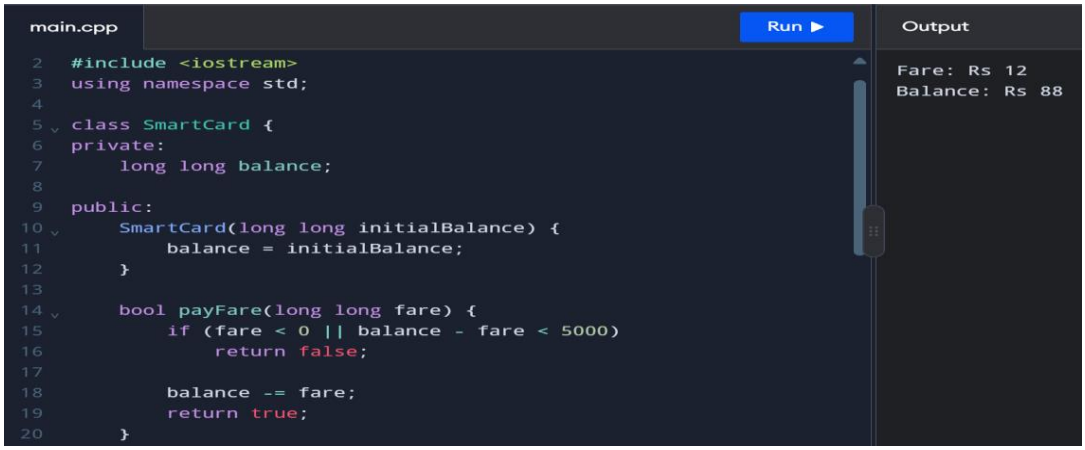
public:

    SmartCard(long long initialBalance) {

        balance = initialBalance; }

    bool payFare(long long fare) {
```

```
        if (fare < 0 || balance - fare < 5000)
            return false;
        balance -= fare;
        return true; }
    long long getBalance() const {
        return balance; }};
int main() {
    SmartCard card(10000);
    long long fare = 1200;
    if (card.payFare(fare)) {
        cout << "Fare: Rs " << fare / 100.0 << '\n';
        cout << "Balance: Rs "
            << card.getBalance() / 100.0 << '\n'; }
 else {
        cout << "Insufficient balance\n"; }
    return 0;}
```

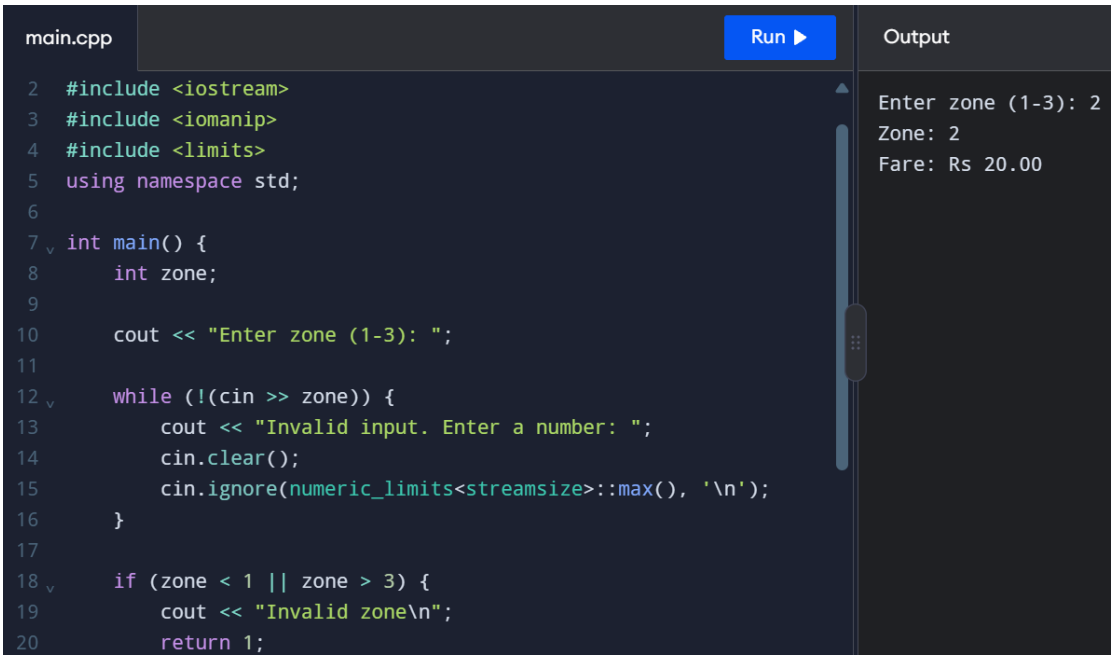


Conclusion: The procedural version is straightforward, but the object-oriented version provides better encapsulation, maintainability and extensibility.

Task 8 — Read and Print Data Safely

The program should validate user input before calculating the fare.

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;
int main() {
    int zone;
    cout << "Enter zone (1-3): ";
    while (!(cin >> zone)) {
        cout << "Invalid input. Enter a number: ";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');}
    if (zone < 1 || zone > 3) {
        cout << "Invalid zone\n";
        return 1; }
    long long fare = zone * 1000LL;
    cout << fixed << setprecision(2);
    cout << "Zone: " << zone << '\n';
    cout << "Fare: Rs " << fare / 100.0 << '\n';
    return 0;}
```



Explanation

- cin reads user input.
- cin.fail() or !(cin >> zone) detects invalid numeric input.
- cin.clear() resets the error state.

- cin.ignore() discards invalid characters.
- <iomanip> provides setw(), setfill(), fixed and setprecision() for formatted reports.

For example, std::fixed << std::setprecision(2) displays a rupee amount with two decimal places.

Alternative: getline() reads a complete line, and stoi() converts it inside a try-catch block. Both approaches take \(O(L)\) time to process an input line of length \(L\). getline() provides more control over validation of the entire line.

Task 9 — Implement and Test the Fare Logic

The complete program is provided below. It includes the fare calculation, balance check, input validation and five sample journeys.

Example:

```
#include <iostream>#include <iomanip>#include <limits>using namespace std;
constexpr long long MIN_BALANCE = 5000;
long long calculateFare(int zone, int hour) {
   long long base;

   switch (zone) {
      case 1: base = 1000; break;
      case 2: base = 2000; break;
      case 3: base = 3000; break;
      default: return -1;
   }

   bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);

   if (peak)
      base += base * 20 / 100;

   return base;}
int main() {
   long long balance = 10000;
   int zone, hour;

   cout << fixed << setprecision(2);
   cout << "Starting balance: Rs "
       << balance / 100.0 << '\n';
```

```
for (int i = 1; i <= 5; i++) {
    cout << "\nJourney " << i << '\n';
    cout << "Enter zone (1-3) and hour (0-23): ";

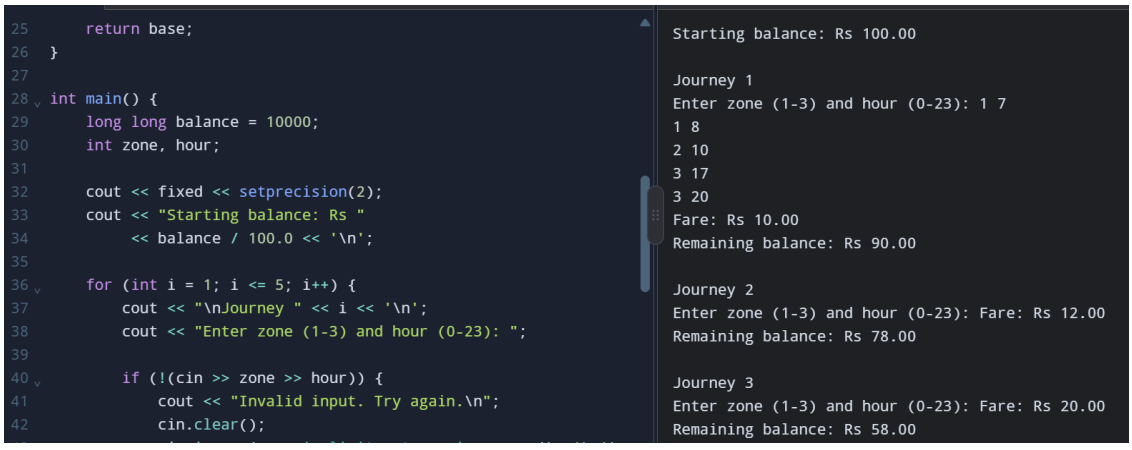
    if (!(cin >> zone >> hour)) {
        cout << "Invalid input. Try again.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        i--;
        continue;
    }

    if (zone < 1 || zone > 3 ||
        hour < 0 || hour > 23) {
        cout << "Invalid zone or hour. Try again.\n";
        i--;
        continue;
    }

    long long f = calculateFare(zone, hour);

    if (balance - f >= MIN_BALANCE) {
        balance -= f;
        cout << "Fare: Rs " << f / 100.0 << '\n';
        cout << "Remaining balance: Rs "
             << balance / 100.0 << '\n';
    } else {
        cout << "Journey denied: Low balance.\n";
        cout << "Balance: Rs " << balance / 100.0 << '\n';
    }
}

return 0;}
```



```
    return base;
}

int main() {
    long long balance = 10000;
    int zone, hour;

    cout << fixed << setprecision(2);
    cout << "Starting balance: Rs "
         << balance / 100.0 << '\n';

    for (int i = 1; i <= 5; i++) {
        cout << "\nJourney " << i << '\n';
        cout << "Enter zone (1-3) and hour (0-23): ";

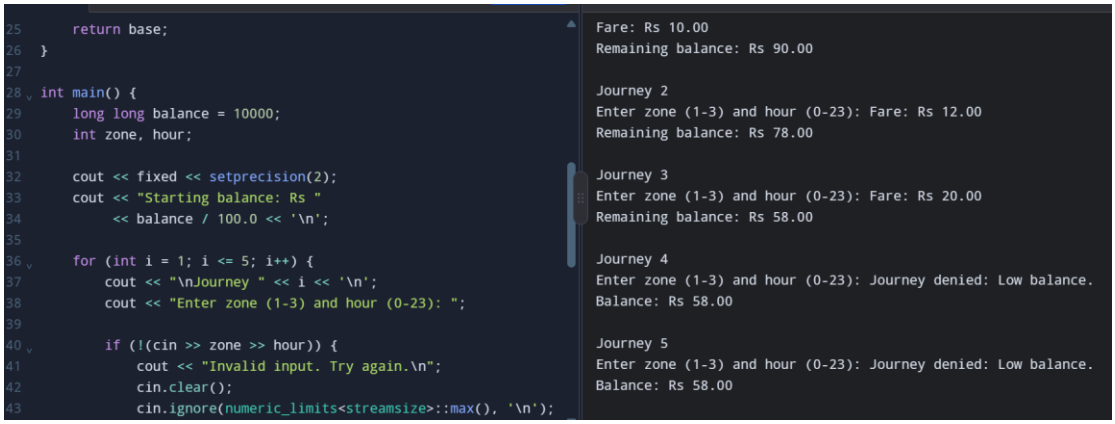
        if (!(cin >> zone >> hour)) {
            cout << "Invalid input. Try again.\n";
            cin.clear();
```

```
Starting balance: Rs 100.00

Journey 1
Enter zone (1-3) and hour (0-23): 1 7
1 8
2 10
3 17
3 20
Fare: Rs 10.00
Remaining balance: Rs 90.00

Journey 2
Enter zone (1-3) and hour (0-23): Fare: Rs 12.00
Remaining balance: Rs 78.00

Journey 3
Enter zone (1-3) and hour (0-23): Fare: Rs 20.00
Remaining balance: Rs 58.00
```



```
    return base;
}

int main() {
    long long balance = 10000;
    int zone, hour;

    cout << fixed << setprecision(2);
    cout << "Starting balance: Rs "
         << balance / 100.0 << '\n';

    for (int i = 1; i <= 5; i++) {
        cout << "\nJourney " << i << '\n';
        cout << "Enter zone (1-3) and hour (0-23): ";

        if (!(cin >> zone >> hour)) {
            cout << "Invalid input. Try again.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```
Fare: Rs 10.00
Remaining balance: Rs 90.00

Journey 2
Enter zone (1-3) and hour (0-23): Fare: Rs 12.00
Remaining balance: Rs 78.00

Journey 3
Enter zone (1-3) and hour (0-23): Fare: Rs 20.00
Remaining balance: Rs 58.00

Journey 4
Enter zone (1-3) and hour (0-23): Journey denied: Low balance.
Balance: Rs 58.00

Journey 5
Enter zone (1-3) and hour (0-23): Journey denied: Low balance.
Balance: Rs 58.00
```

Test table

Zone	Hour	Expected fare
Inner	7	₹10.00
Inner	8	₹12.00
Middle	10	₹20.00
Outer	17	₹36.00
Outer	20	₹30.00

The program should also be tested with an unknown zone, an invalid hour, letters instead of numbers, and a balance that would fall below ₹50.

Complexity: Each fare calculation takes \(O(1)\) time. Processing \(n\) journeys takes \(O(n)\) time.

Task 10 — Compare Expectations with Execution

Before running a program, predict its output by evaluating expressions according to C++ rules.

The program should then be compiled and executed, and the predicted output compared with the actual output.

Differences may result from:

- Operator precedence.
- Integer division and remainders.

- Post-increment and pre-increment.
- Implicit type conversions.
- Loop conditions and update expressions.
- Conditional operator evaluation.

Example: For `10 / 3`, the expected result is `3`, not `3.33333`, because both operands are integers.

A proper report should contain the predicted output, actual output, differences, and the C++ rule explaining each difference.

Task 11 — Final Recommendation

The recommended design is to retain the core smart-card system while improving its validation and fare-management logic.

The system should:

- Use constexpr constants for fares and limits.
- Store money in integer paise.
- Validate zones, hours and user input.
- Use a SmartCard class to protect the balance.
- Apply the peak surcharge only during the specified hours.
- Maintain test cases for normal, boundary and invalid inputs.

Conclusion: The revised design is more reliable because it prevents invalid journeys, avoids floating-point fare errors and makes future fare changes easier to maintain.

Task 12 — Re-evaluate and Adapt the Design

The new requirements are:

- Add an Airport zone with a base fare of ₹60.
- Introduce a 10% weekend discount that does not combine with the peak surcharge.
- Add a pass allowing unlimited journeys in two selected zones.

A. Impact Analysis

The following components need changes:

- Add Airport to the Zone enum.
- Add a ₹60 fare to the fare table.
- Modify the fare calculation to accept weekend status.
- Ensure the weekend discount takes priority over the peak surcharge.
- Add a Pass class containing two selected zones and a validity check.

B. Evaluate the Original Design

The original switch can support a fourth zone by adding another case. However, repeated changes become harder to manage as the number of zones and fare rules grows.

A fare table is easier to extend. The original SmartCard class can remain, while a separate Pass class handles unlimited travel within the selected zones.

C. **Adapted C++ Design**

```
#include <iostream>using namespace std;
enum class Zone {
   Inner = 0,
   Middle = 1,
   Outer = 2,
   Airport = 3};
constexpr long long FARES[] = {
   1000, 2000, 3000, 6000};
long long calculateFare(
   Zone zone, bool peak, bool weekend) {

   int index = static_cast<int>(zone);

   if (index < 0 || index > 3)
      return -1;

   long long amount = FARES[index];

   if (weekend) {
      amount -= amount * 10 / 100;
   } else if (peak) {
      amount += amount * 20 / 100;
```

```
    }

    return amount;}
class Pass {private:
    Zone first;
    Zone second;
public:
    Pass(Zone a, Zone b) : first(a), second(b) {}

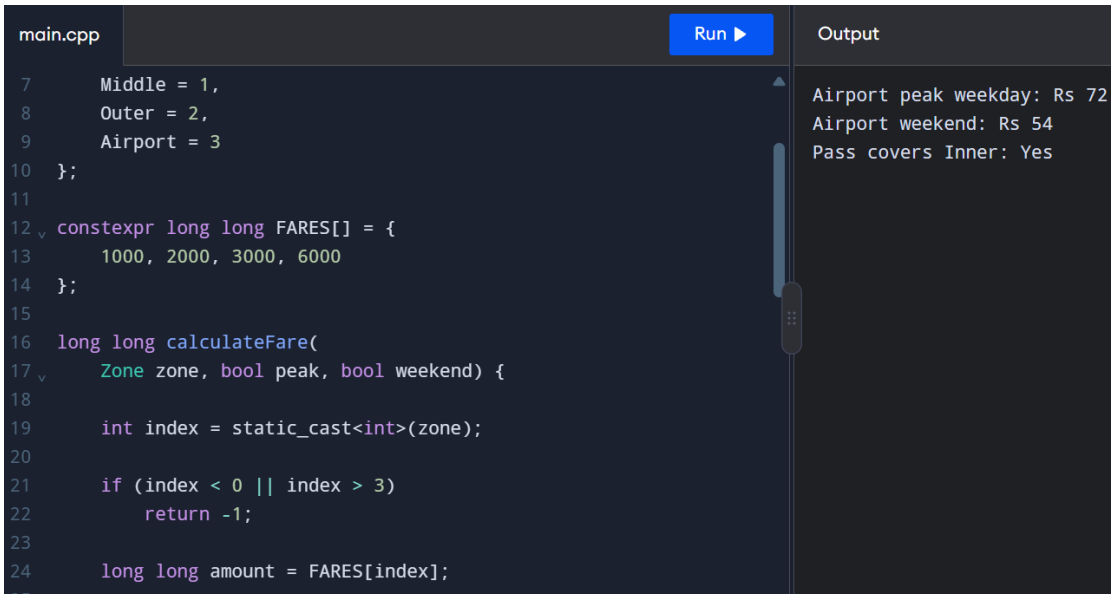
    bool covers(Zone z) const {
        return z == first || z == second;
    }};
int main() {
    cout << "Airport peak weekday: Rs "
         << calculateFare(Zone::Airport, true, false) / 100.0
         << '\n';

    cout << "Airport weekend: Rs "
         << calculateFare(Zone::Airport, true, true) / 100.0
         << '\n';

    Pass p(Zone::Inner, Zone::Middle);

    cout << "Pass covers Inner: "
         << (p.covers(Zone::Inner) ? "Yes" : "No")
         << '\n';

    return 0;}
```



D. Evidence-Based Defence

The revised design is preferable because the fare table supports four zones without adding multiple branches to the fare calculation. Weekend and peak rules are mutually exclusive, so a weekend journey receives the 10% discount rather than the peak surcharge.

The Pass class separates unlimited-zone access from ordinary fare deduction. Tests should verify all four zones, peak and weekend calculations, invalid zones, pass-covered journeys and journeys outside the pass.

Complexity: Fare lookup and pass validation both take $O(1)$ time. The fare table uses $O(z)$ space for z zones.